

Fast Neuroevolution for Flappy Bird: A Headless Simulator and PRML-Style Analysis of a Simple Neuron Policy

Name: Angel Mencia

COGS 118B

Virginia De Sa

Date: December 12, 2025

Abstract

I trained an AI to play Flappy Bird using a fast “headless” evolution loop. At first, I used a Pygame version for visuals, but it was slow for big experiments because the simulation speed was tied to rendering. So I made `fast_train_plus`: it keeps the same basic physics/collisions, but removes drawing and adds logging so I can actually run hundreds of generations and make clean graphs. The controller is intentionally simple: it takes a few normalized “vision” inputs about the next pipe (2–3 distance features) plus a bias input fixed at 1, computes a weighted sum, and flaps when the output crosses a threshold. I evaluate learning using learning curves (fitness vs generation), fixed-seed evaluation for reproducibility, and population-level statistics.^{[2][1][7]}

(NEAT inspiration: add-connection / add-node diagram)

Source to screenshot: Stanley & Miikkulainen (2002) NEAT paper PDF

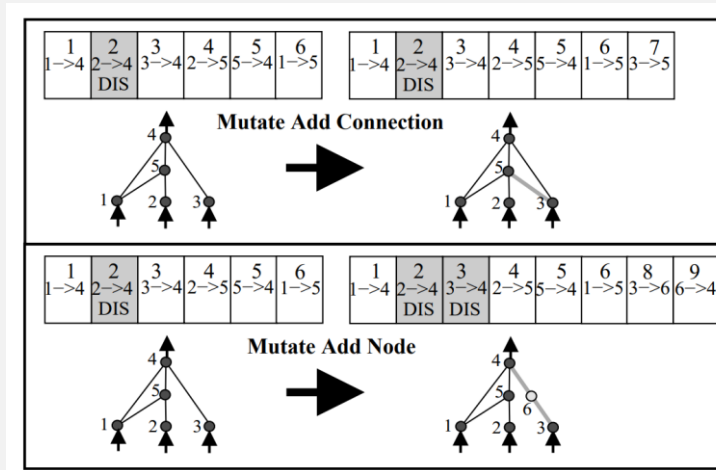


Figure 1. NEAT-style mutation idea (used as inspiration, not fully implemented). [1]

1. Introduction

Flappy Bird looks simple, but it's a surprisingly unforgiving control problem: at every moment the agent decides "flap or don't flap," and a small timing error can immediately end the run. I used it as a testbed because the rules are easy to define, yet performance still shows a clear learning curve over training.

The Project was a standard Pygame game loop since it's great for visualization and demos. The downside is that it's not ideal for experiments. When the goal is something like 200 generations $\times$ 20 birds $\times$ thousands of steps, rendering quickly becomes the bottleneck and it's harder to collect clean logs.

This is where the project became more than "just Flappy Bird." I was inspired by the main idea behind NEAT-style neuroevolution: instead of training with gradients, you can improve a population of policies through selection, inheritance, and mutation. Rather than using a full NEAT library, I implemented a simpler evolution loop from scratch and built a headless trainer (fast_train_plus) that keeps the same physics and collision logic but removes drawing. This made it practical to run fast trials, repeat runs with fixed seeds, and produce PRML-style plots that show learning and stability.

1.1 The “simple neuron” controller (what a bird actually does)

Each bird’s brain is basically one small neuron (perceptron-style). It gets a small vision vector about the next pipe and a bias input. Then it does a weighted sum and runs an activation. If the output is bigger than a threshold, the bird flaps. So you can think of “flap” like the neuron firing.^{[3][4][8]}

Conceptual decision rule:

$$a = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + w_{\text{bias}} \cdot 1 \quad \text{and} \quad \text{if } \text{output}(a) > \tau \rightarrow \text{flap}$$

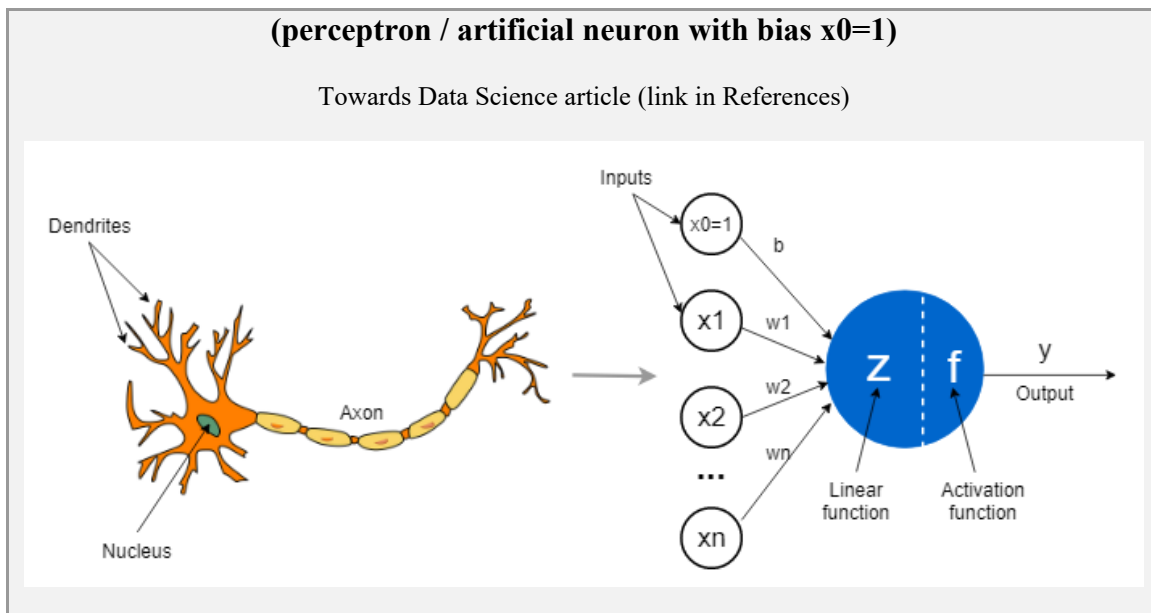


Figure 2. Perceptron-style neuron with bias input (matches “vision + bias $\rightarrow$ threshold flap” setup). [8]

2. Motivation and related work

This project started because I saw that neuroevolution can work surprisingly well for games. I used Max Rohowsky’s open-source project as a starting inspiration for structure and ideas, but I wanted to understand what was happening and make my own fast training + analysis pipeline.^[7]

My goal wasn’t to perfectly re-create full NEAT. Instead, I tried to keep the policy small and understandable, then focus on what PRML emphasizes: repeatable experiments, clean plots, and actually measuring behavior over time.^[2]

3. Methods

3.1 fast_train_plus: headless training loop

fast_train_plus runs the same basic Flappy Bird physics and collisions, but without any rendering. Each generation does: (1) simulate every bird to get a fitness score, (2) select the better birds, (3) make a new population by cloning + mutating weights, and (4) log everything to disk.

Run configuration used for the main results in this paper:

```
python fast_train_plus.py --seed 0 --gens 200 --pop 20 --max_steps 4000 --  
logdir runs/trial_seed0 --save_pop_every 1 --save_policy_every 1 --eval_every  
1 --eval_seeds 0,1,2,3,4
```

3.2 What the key command-line args control

- --seed: makes the whole run repeatable (same random sequence).
- --gens: number of generations (how many evolutionary updates).
- --pop: number of birds per generation.
- --max_steps: maximum steps per episode. With fitness = steps survived, this also caps the max fitness.
- --logdir: where logs and checkpoints get saved.
- --save_pop_every / --save_policy_every: how often to save snapshots.
- --eval_every and --eval_seeds: evaluate the best bird on fixed seeds (helps show generalization).

The main training loop. Each generation we simulate the whole population, calculate best/average fitness, optionally run a fixed-seed evaluation, save snapshots, and print the learning progress (gen=... best=... avg=...).

3.3 Core code screenshots (4 total)

File: fast_train_plus.py

Why: this is the “learning over generations” part

File: fast_train_plus.py

[illegible]

```

211 while alive > 0 and steps < max_steps:
212     steps += 1
213
214     pipe = closest_pipe()
215     prev_passed = pipe.passed
216     pipe.step()
217
218     # # pipes passed for alive birds
219     if (not prev_passed) and pipe.passed:
220         for pl in players:
221             if pl.alive:
222                 pl.pipes_passed += 1
223
224     # Respawn keep ONE pipe only
225     if pipe.x + PIPE_W < 0:
226         config.pipes[0].respawn()
227         pipe = config.pipes[0]
228
229     gap_y = gap_center_y(pipe)
230
231     for pl in players:
232         if not pl.alive:
233             continue
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337

```

```

54 pygame.draw.rect(window, self.species_color, s
55
56
57 def ground_collision(self, ground):
58     return pygame.Rect.colliderect(self.rect, grou
59
60 def sky_collision(self):
61     return bool(self.rect.y < 30)
62
63 def pipe_collision(self):
64     for p in config.pipes:
65         return pygame.Rect.colliderect(self.rect,
66             pygame.Rect.colliderect(self.rect,
67
68 def update(self, ground):
69     if not (self.ground_collision(ground) or self.
70         # Gravity
71         self.vel += 0.25
72         self.rect.y += self.vel
73         if self.vel > 5:
74             self.vel = 5
75         # Increment lifespan
76         self.lifespan += 1
77     else:
78         self.alive = False

```

```

512 with gen_stats_path.open("a", newline="", encoding="utf-8") as f:
513     w = csv.writer(f)
514     w.writerow([
515         g,
516         best_fit, avg_fit, med_fit, std_fit,
517         best_pipes, avg_pipes,
518         best_fr, avg_fr,
519         best_ge, avg_ge,
520         div["w_dim"], div["w_var_mean"], div["w_std_mean"],
521         frac_capped,
522         test_mean_fit, test_std_fit,
523         test_mean_pipes, test_std_pipes,
524         elapsed
525     ])
526
527 if args.save_pop_every and (g % args.save_pop_every == 0):
528     maybe_save_population_snapshot(logdir, g, players)
529
530 if args.save_policy_every and (g % args.save_policy_every == 0):
531     maybe_save_policy_snapshot(logdir, g, best_player, grid_n=args.policy_grid)
532
533 players, best = evolve(players, pop_size=args.pop)
534 if best_ever is None or best.fitness > best_ever.fitness:
535     best_ever = best
536
537 print(f"gen={g:4d} best={best_fit:6.1f} avg={avg_fit:6.1f} elapsed={elapsed:6.2f}s")

```

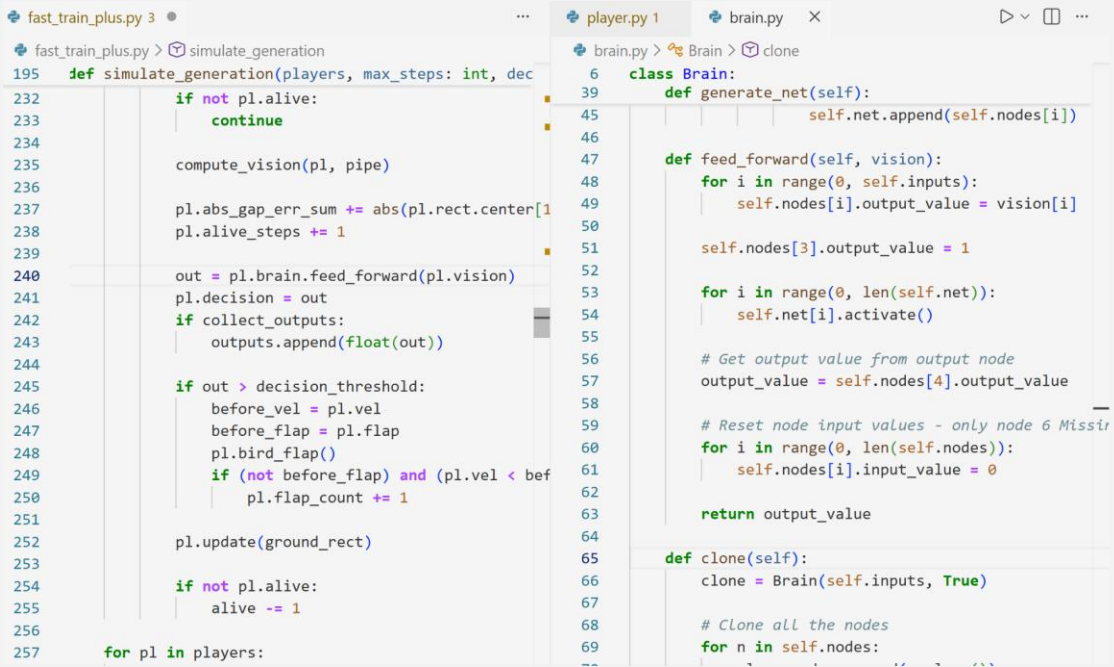
Why: proves the simulator is headless + fast

The headless simulation loop: physics, pipes, collisions, and fitness.

The headless simulator step loop. This is where the run is actually “played”: pipes move, the bird’s vision is updated, decisions are made, and then physics + collisions are applied through the player update (gravity, velocity, position, and death on collision).

File: fast_train_plus.py OR brain.py

Capture: vision inputs + bias + weighted sum + threshold flap



```

fast_train_plus.py
195 def simulate_generation(players, max_steps: int, dec
232     if not pl.alive:
233         continue
234
235     compute_vision(pl, pipe)
236
237     pl.abs_gap_err_sum += abs(pl.rect.center[1
238     pl.alive_steps += 1
239
240     out = pl.brain.feed_forward(pl.vision)
241     pl.decision = out
242     if collect_outputs:
243         outputs.append(float(out))
244
245     if out > decision_threshold:
246         before_vel = pl.vel
247         before_flap = pl.flap
248         pl.bird_flap()
249         if (not before_flap) and (pl.vel < bef
250             pl.flap_count += 1
251
252     pl.update(ground_rect)
253
254     if not pl.alive:
255         alive -= 1
256
257     for pl in players:

```

```

brain.py
6 class Brain:
39     def generate_net(self):
45         self.net.append(self.nodes[i])
46
47     def feed_forward(self, vision):
48         for i in range(0, self.inputs):
49             self.nodes[i].output_value = vision[i]
50
51         self.nodes[3].output_value = 1
52
53         for i in range(0, len(self.net)):
54             self.net[i].activate()
55
56         # Get output value from output node
57         output_value = self.nodes[4].output_value
58
59         # Reset node input values - only node 6 Missir
60         for i in range(0, len(self.nodes)):
61             self.nodes[i].input_value = 0
62
63         return output_value
64
65     def clone(self):
66         clone = Brain(self.inputs, True)
67
68         # Clone all the nodes
69         for n in self.nodes:

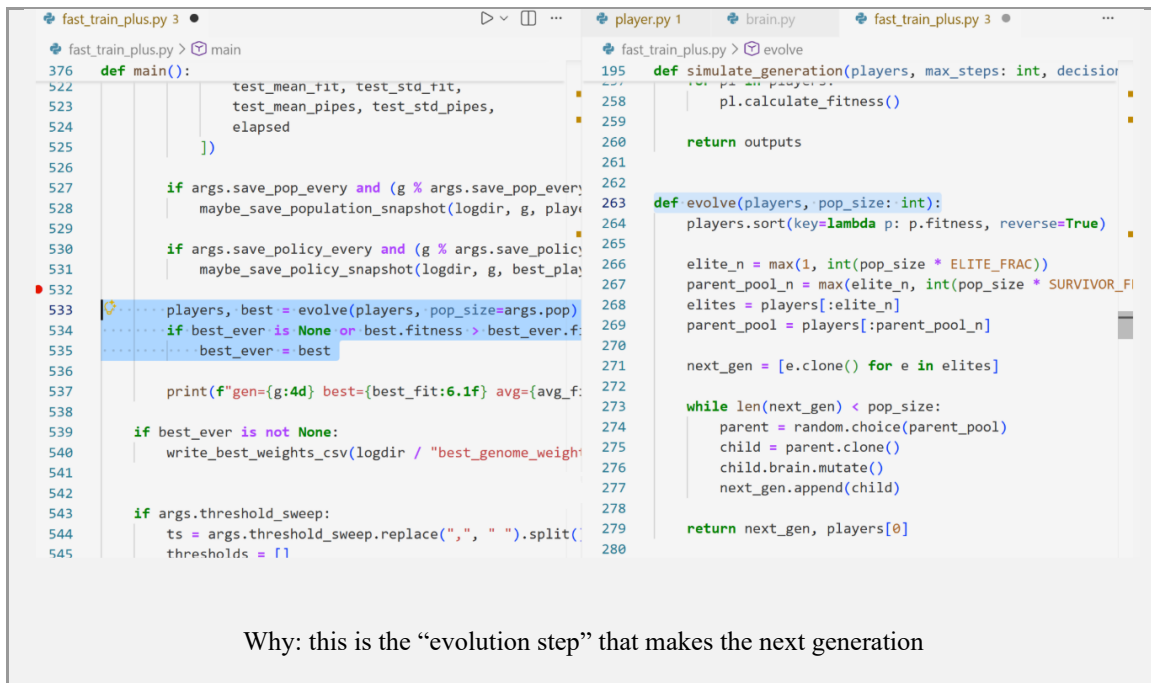
```

Why: connects directly to Figure 2 (perceptron)

The perceptron-like policy: vision + bias → output → flap threshold.

The “simple neuron” decision. We feed the bird’s vision inputs plus a bias input fixed at 1 into a forward pass, and if the output crosses a threshold, the bird flaps — basically like a neuron “firing.”

File: fast_train_plus.py



```
fast_train_plus.py 3
def main():
    test_mean_fit, test_std_fit,
    test_mean_pipes, test_std_pipes,
    elapsed
    ])
    if args.save_pop_every and (g % args.save_pop_every):
        maybe_save_population_snapshot(logdir, g, playe
    if args.save_policy_every and (g % args.save_policy):
        maybe_save_policy_snapshot(logdir, g, best_play
    532 players, best = evolve(players, pop_size=args.pop)
    534 if best_ever is None or best.fitness > best_ever.f
    535 best_ever = best
    536
    print(f'gen={g:4d} best={best_fit:6.1f} avg={avg_f:
    538
    if best_ever is not None:
        write_best_weights_csv(logdir / "best_genome_weigh
    541
    542
    if args.threshold_sweep:
        ts = args.threshold_sweep.replace(",", " ").split(
        thresholds = []

player.py 1
def evolve(players, max_steps: int, decision
    pl.calculate_fitness()
    return outputs
    261
    262
    263 def evolve(players, pop_size: int):
    264     players.sort(key=lambda p: p.fitness, reverse=True)
    265
    266     elite_n = max(1, int(pop_size * ELITE_FRAC))
    267     parent_pool_n = max(elite_n, int(pop_size * SURVIVOR_FI
    268     elites = players[:elite_n]
    269     parent_pool = players[:parent_pool_n]
    270
    271     next_gen = [e.clone() for e in elites]
    272
    273     while len(next_gen) < pop_size:
    274         parent = random.choice(parent_pool)
    275         child = parent.clone()
    276         child.brain.mutate()
    277         next_gen.append(child)
    278
    279     return next_gen, players[0]
    280
```

Why: this is the “evolution step” that makes the next generation

Selection + mutation that creates the next generation

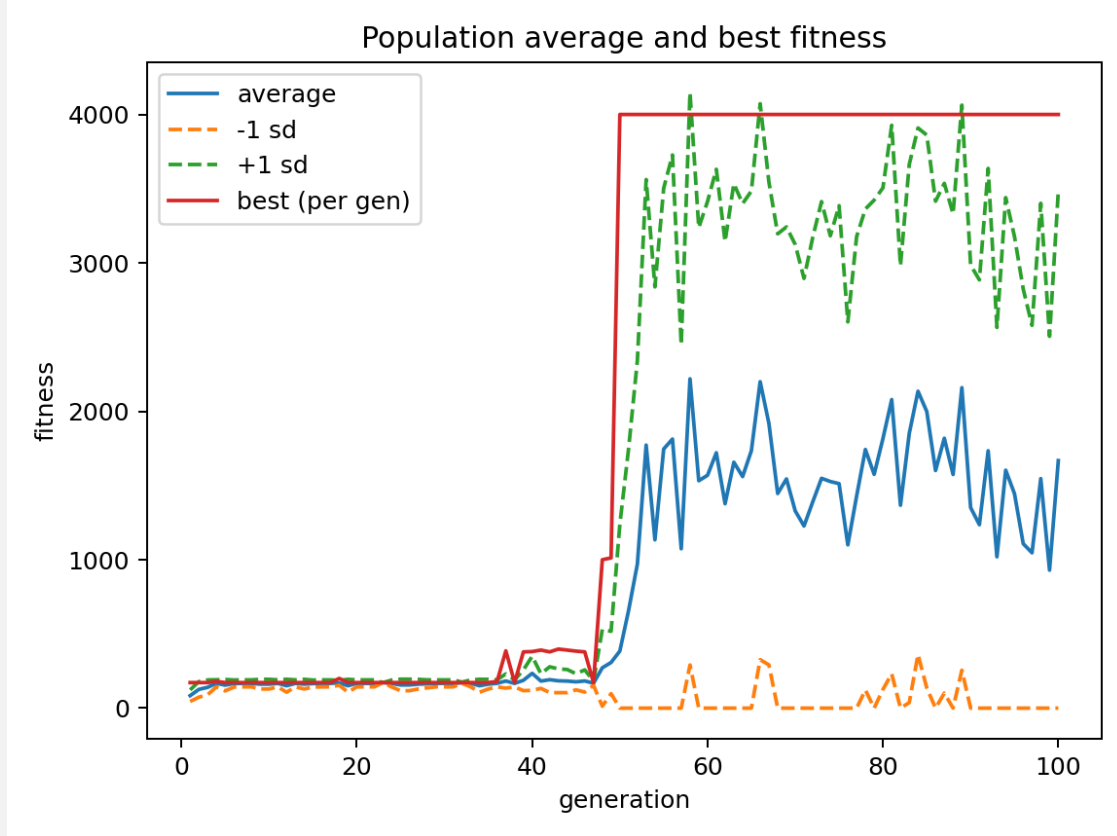
The evolution update. After each generation, we build the next population by keeping the best birds (elite/selection), cloning them, and mutating their weights to create new birds for the next generation..

4. Results

For the run with seed 0, performance jumped pretty quickly and hit the ceiling at `max_steps = 4000` around generation ~50. After that, the best bird kept getting 4000, which basically means “it survived the whole episode every time” under this cap.

I plot the results in two ways: (1) a full learning curve to show the big picture, and (2) zoomed-in curves (first 30–60 generations) to show the actual “breakthrough moment” where a bird first discovers a stable strategy.

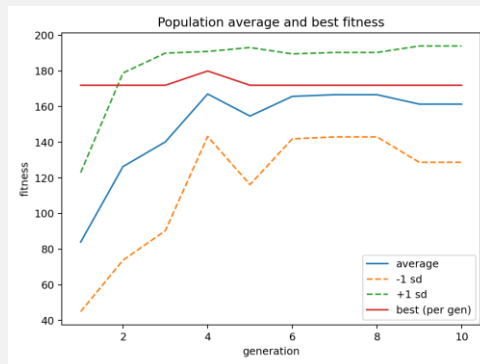
(Learning curve: generations 1–200)

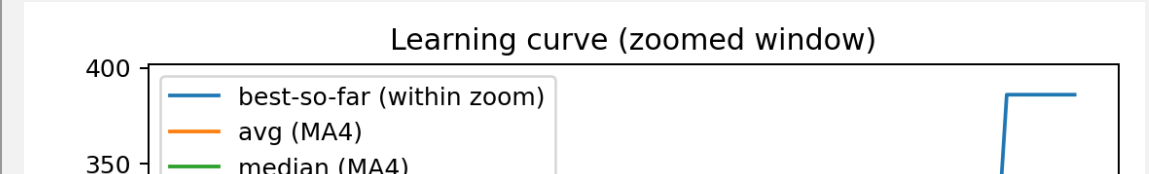


This plot reflects the same high-level behavior you see in NEAT-style neuroevolution: a population explores many policies, selection preserves strong individuals, and mutation introduces variation. The “best-per-generation” curve can improve rapidly once a good controller is discovered, while the population average remains noisy because most offspring are mutated variants that may perform worse. The wide ± 1 standard deviation band shows persistent diversity in the population, which NEAT explicitly manages through speciation; in my simplified version, diversity appears naturally from mutation and the fact that not all individuals are elite.

Figure 3. Learning curve over generations (big picture).

(Zoomed learning curve: generations 1–30 or 1–60)





Goal: show the first time best fitness hits 4000 around gen ~50

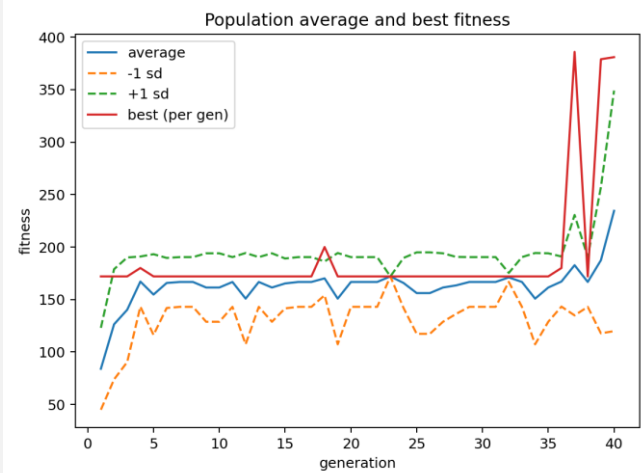
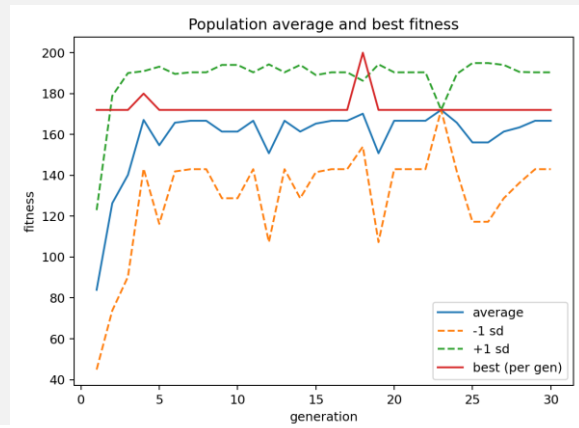


Figure 4. Zoomed learning curve (shows the breakthrough).

(Fixed-seed evaluation: eval_seeds 0–4)

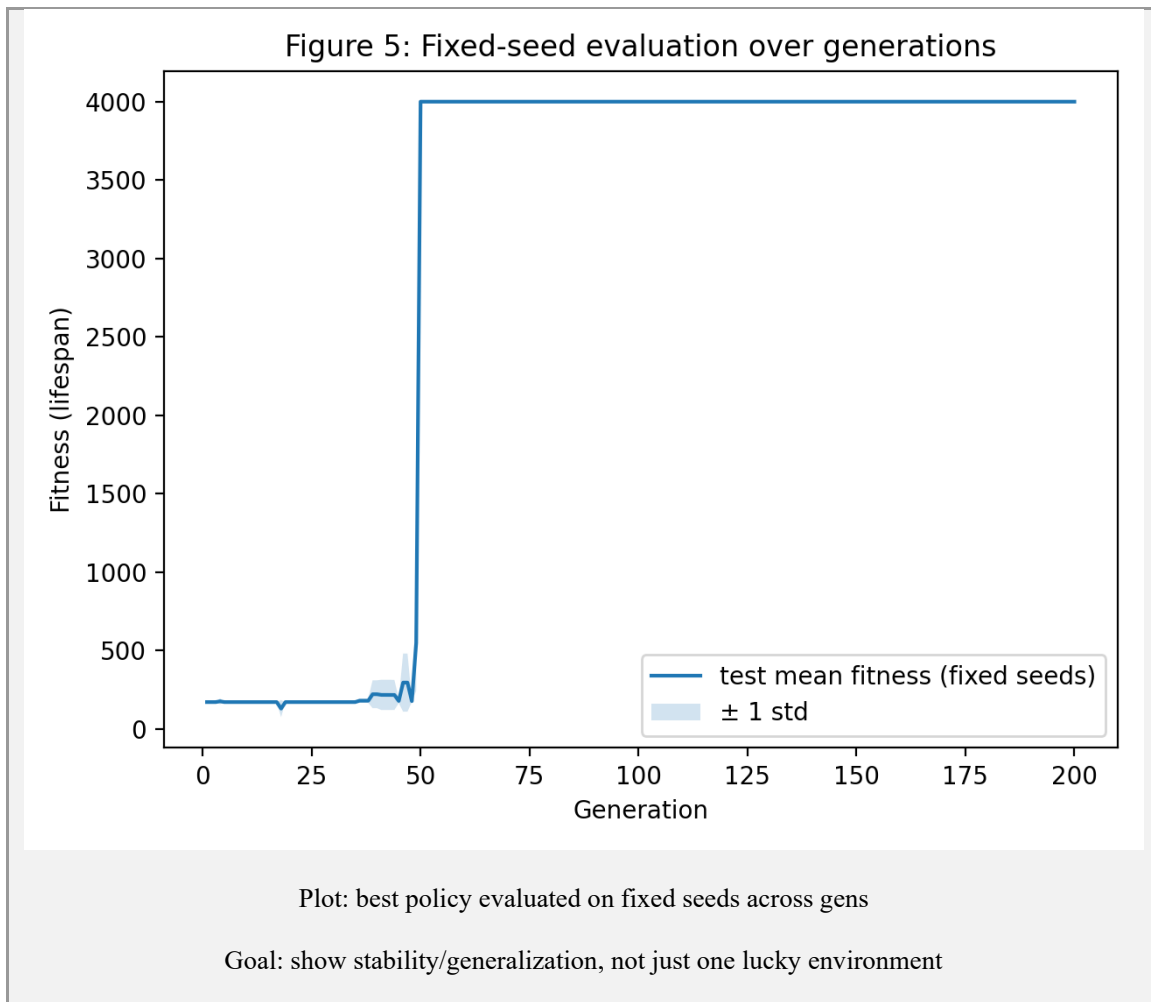


Figure 5. Fixed-seed evaluation curve across training (generalization check).

5. Discussion

The biggest win in this project was speed. Once the simulator was headless, I could run lots of generations quickly and log everything in a consistent way. That made it much easier to do the kind of analysis emphasized in PRML—repeatable trials, learning curves, and comparisons across random seeds—instead of relying on a single demo run.

One thing I had to be careful about is what “convergence” really means in this setup. Because fitness is capped by `max_steps`, hitting a score of 4000 does not prove the policy is perfect in general—it only shows it can survive for the full time limit I allowed. Increasing `max_steps` makes the evaluation stricter and can reveal whether a policy is truly stable or just good enough to pass the current horizon.

If I had more time, the first extension would be improving the input features. For example, adding the bird's vertical velocity, the center of the upcoming gap, or a more direct "error-to-gap-center" feature would likely make learning more reliable and reduce randomness. The second extension would be stronger generalization testing: evaluating on more seeds, widening the range of pipe gaps/spacings, or training on one distribution and testing on a slightly different one to see how well the controller transfers.

References

1. Stanley, K. O., & Miikkulainen, R. (2002). Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation*, 10(2), 99–127.
<https://nn.cs.utexas.edu/downloads/papers/stanley.ec02.pdf>
2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. Perceptron (Wikipedia). <https://en.wikipedia.org/wiki/Perceptron>
4. Artificial neuron (Wikipedia). https://en.wikipedia.org/wiki/Artificial_neuron
5. scikit-learn example: Plotting learning curves. https://scikit-learn.org/stable/auto_examples/model_selection/plot_learning_curve.html
6. Nielsen, M. (2015). *Neural Networks and Deep Learning*, Chapter 1.
<https://neuralnetworksanddeeplearning.com/chap1.html>
7. Rohowsky, M. *neft-flappy-bird* (GitHub). <https://github.com/maxrohowsky/neft-flappy-bird?tab=readme-ov-file#libraries>
8. Towards Data Science. The concept of artificial neurons (perceptrons) in neural networks. <https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/>